

FPGA Design and Implementation of a Low-Power Systolic Array-Based Adaptive Viterbi Decoder

Man Guo, M. Omair Ahmad, *Fellow, IEEE*, M. N. S. Swamy, *Life Fellow, IEEE*, and Chunyan Wang, *Senior Member, IEEE*

Abstract—In this paper, by modifying the well-known Viterbi algorithm, an adaptive Viterbi algorithm that is based on strongly connected trellis decoding is proposed. Using this algorithm, the design and a field-programmable gate array implementation of a low-power adaptive Viterbi decoder with a constraint length of 9 and a code rate of 1/2 is presented. In this design, a novel systolic array-based architecture with time multiplexing and arithmetic pipelining for implementing the proposed algorithm is used. It is shown that the proposed algorithm can reduce by up to 70% the average number of ACS computations over that by using the non-adaptive Viterbi algorithm, without degradation in the error performance. This results in lowering the switching activities of the logic cells, with a consequent reduction in the dynamic power. Further, it is shown that the total power consumption in the implementation of the proposed algorithm can be reduced by up to 43% compared to that in the implementation of the nonadaptive Viterbi algorithm, with a negligible increase in the hardware.

Index Terms—Adaptive Viterbi decoder, field-programmable gate array (FPGA) implementation, low-power design, systolic array architecture.

I. INTRODUCTION

CONVOLUTIONAL codes and the Viterbi algorithm [1] are known to provide a strong forward error correction (FEC) scheme, which has been widely utilized in digital communication applications. It has been shown that the larger the constraint length K used in a convolutional encoding process, the more powerful the code produced. Thus, a convolutional code with a relatively large constraint length of $K = 9$ is used for the IS-95 code-division multiple access (CDMA) wireless communication standard [2]. However, the complexity of the Viterbi decoding process becomes very large for a constraint length K of this size. As a result, it would be difficult to achieve a hardware implementation of a Viterbi decoder for $K = 9$, in order to meet the requirements of the power, speed and area. In recent years, Viterbi decoders have been mostly used in mobile

systems that require portable battery operations, thus making the power consumption a critical concern to the designers. Some attempts have been made for low-power design of Viterbi decoders with $K = 9$. The field-programmable gate array (FPGA) designs of the decoders in [3] and [4], and the application-specific integrated circuit (ASIC) design of the decoder in [5] are based on a serial butterfly architecture. Another ASIC design of the decoder proposed in [6] is based on a parallel butterfly architecture with bit-serial arithmetic. At present, there is no implementation for a large constraint-length Viterbi decoder using the systolic array architecture suggested by Chang and Yao [7]. The systolic array architecture in [7] can provide a moderate speed over the serial and parallel architectures due to its high degree of pipelining and multiprocessing. It can also provide, for different convolutional code structures, a general method for a simple adjacent cell interconnection. However, to design the Viterbi decoder for decoding a convolutional code with a large value of K , say, $K = 9$ and a code rate of $r = 1/2$, the complexity of the computations will render the adoption of the systolic array architecture in [7] not to be feasible.

As presented in [7], the Viterbi algorithm can be formulated employing a matrix–vector multiplication, whose hardware implementation can be realized using a systolic array architecture [8]–[10]. Since the low-connectivity trellis decoding results in the adjacency matrix to become very sparse, the efficiency of the hardware utilization is rather poor in this formulation. A strongly connected trellis-based Viterbi decoding method was also proposed in [7] for improving the utilization of the systolic array processors as well as for increasing the throughput of the decoding. A strongly connected trellis is obtained by merging the stages in each group of the $(K - 1)$ contiguous stages in the original low-connectivity trellis so that every state in the $(K - 1)$ th stage of the group is reachable from the states of the preceding $(K - 2)$ stages. This, however, makes the amount of the addition, comparison, and selection (ACS) computations in the strongly connected Viterbi decoding process to become very excessive. For instance, in the Viterbi decoder, for $K = 9$ and $r = 1/2$, the path metric update at each state has 256 ACS computations. Since for a trellis with $K = 9$, there are 256 states in a strongly connected trellis, a total of 256×256 ACS computations for each stage are required. This contrasts with a total of $2 \times 256 \times 8$ ACS computations for each group of 8 stages in a two-ACS per state computation of the corresponding low-connectivity trellis decoding. Thus, a major drawback of the strongly connected trellis Viterbi decoding is its increased computational complexity, in that its number of ACS computations per stage is 16 times higher than that in the corresponding

Manuscript received October 31, 2002; revised May 19, 2004. This work was supported by the Natural Sciences and Engineering Research Council (NSERC) of Canada, the MICRONET National Network of Centers of Excellence, and Fonds pour la Formation de Chercheurs et l'Aide à la Recherche (FCAR) of Québec. This paper was recommended by Associate Editor J. McCanny.

M. Guo was with the Center for Signal Processing and Communications, Department of Electrical and Computer Engineering, Concordia University, Montréal, QC H3G 1M8, Canada. She is now with 1021 Technologies, Kanata, ON, Canada (e-mail: m_guo@ece.concordia.ca).

M. O. Ahmad, M. N. S. Swamy, and C. Wang are with the Center for Signal Processing and Communications, Department of Electrical and Computer Engineering, Concordia University, Montréal, QC H3G 1M8, Canada (e-mail: omair@ece.concordia.ca; swamy@ece.concordia.ca; chunyan@ece.concordia.ca).

Digital Object Identifier 10.1109/TCSI.2004.838266

low-connectivity counterpart. Obviously, this excessive amount of ACS computations will result in a large power consumption in hardware implementation.

Furthermore, the number of pipeline stages (and hence the number of array processors) in the architecture presented in [7] increases exponentially with K . For instance, for decoding a convolutional code with $K = 9$ and $r = 1/2$, there should be 256 processors in the corresponding 256 pipeline stages in both the systolic array for generating the composite branch metrics and the systolic array for processing the path metrics. As a result, the design will be excessively hardware-resource intensive and power consuming, and at the same time, clock-to-data skews caused by the deep pipelining will become critical for proper functioning of the system. In addition, the processing times of the two operations, namely the composite branch metric generation and the path metric update, will differ significantly as K becomes large. For example, with $K = 9$, the eight Euclidean or Hamming distances corresponding to the eight branches in the original low connectivity trellis diagram should be computed and added together for generating a composite branch metric, whereas there are only three operations (ACS) in updating a path metric. Consequently, it is a challenging task to ensure that the two types of processors with different processing times can be synchronized, without slowing down the clock-rate.

In this paper, an adaptive Viterbi algorithm based on strongly connected trellis decoding is proposed to reduce the amount of ACS computation in the Viterbi decoding process. Based on this algorithm, the design and implementation of an adaptive Viterbi decoder for decoding a convolutional code with a constraint length of 9, and a code rate of 1/2 using the Xilinx Virtex-II XC2V1000-4FG256 chip is presented. Some of the results are included in [11]. In this design, a novel systolic array architecture with time multiplexing and arithmetic pipelining is investigated to implement the proposed algorithm. A process for spurious toggle reduction and a clock-to-data skews tolerance scheme are proposed to respectively reduce the power dissipation and avoid the timing violations in the design. Simulation results show that the proposed adaptive Viterbi algorithm reduces the number of ACS computations significantly while maintaining almost the same error performance as that of the nonadaptive Viterbi algorithm. This reduction in the amount of computations results in a substantial reduction in the overall power consumption compared to the implementation of the non-adaptive Viterbi algorithm.

II. MODIFIED ADAPTIVE VITERBI ALGORITHM

It is known that in a Viterbi algorithm all possible distinct paths in the trellis are evaluated at each state and the most likely one with the shortest Hamming or Euclidean distance is determined. Consequently, for the strongly connected trellis Viterbi decoding, the amount of ACS computations increases exponentially as the constraint length K increases. In order to reduce the amount of computations in the Viterbi decoding process, an adaptive Viterbi algorithm was proposed by Chan and Haccoun in 1997 [12]. This algorithm is a modified version of the conventional Viterbi algorithm and is based on low-connectivity

trellis decoding. A sorting operation is used to determine the most likely survivor paths among the paths satisfying the discarding threshold [12]. In 2002, Swaminathan *et al.* [13] introduced a novel dynamically reconfigurable adaptive Viterbi decoder, but did not include a study on the power requirements. Subsequently, Henning and Chakrabarti [14] proposed a method of adaptively approximating the Viterbi decoder for reducing the power requirement.

In this section, the adaptive Viterbi algorithm presented in [12] is modified so that it is suitable for a systolic array-based hardware implementation of the algorithm. The most important modification introduced in the proposed algorithm is the employment of strongly connected trellis decoding technique. This is done in order to achieve an increased utilization of the hardware resources offered by the systolic array architecture and a higher throughput of the decoding. A byproduct of this change is in the elimination of the sorting operation needed in the low-connectivity technique of [12]. Similar to the Viterbi decoding process, the proposed adaptive Viterbi algorithm can be viewed as a process of finding the shortest path in a strongly connected trellis diagram. This process includes mainly the generation of the composite branch metrics, updating of the path metrics, and the traceback operation. In this section, the formulation for these processes is presented for the systolic array-based implementation of the algorithm.

A. Generation of Composite Branch Metrics

Let K be the constraint length and $r = 1/n$ be the code rate of the given low connectivity trellis diagram. By combining $(K - 1)$ branches of the low-connectivity trellis diagram into one single branch, we can convert it into a strongly connected trellis diagram; the single branch in the latter is called a composite branch [7]. Let $b_q(i, j)$ represent the composite branch metric from state i of stage $q - 1$ to state j of stage q of the strongly connected trellis diagram, and $\mathbf{C}_{ij}$ a binary vector of $n(K - 1)$ elements, representing the codeword sequence corresponding to the composite branch. Let $\mathbf{y}_q$ be a quantized vector of $n(K - 1)$ elements, representing the received real-valued sequence at stage q . It has been shown in [7] that under the maximum likelihood criterion the evaluation of $b_q(i, j)$ can be expressed as

$$b_q(i, j) = f(\mathbf{y}_q, \mathbf{C}_{ij}) \quad (1)$$

where f is the likelihood function of $\mathbf{y}_q$ and $\mathbf{C}_{ij}$. For example, f evaluates the Euclidean distance between $\mathbf{C}_{ij}$ and $\mathbf{y}_q$ with soft demodulator decisions, and computes the Hamming distance between $\mathbf{C}_{ij}$ and $\mathbf{y}_q$ with hard demodulator decisions. In this paper, binary phase shift keying (BPSK) modulation and soft demodulator decisions are assumed. Under the BPSK modulation, the minimum and maximum amplitude values, $-a$ and a , can be used to respectively represent the symbol values “1” and “0” of a binary bit in $\mathbf{C}_{ij}$. The codeword symbol $x[k]$ can then be expressed as

$$x[k] = \{2C_{ij}[k] - 1\} a, \quad k = 0, 1, \dots, n(K-1)-1 \quad (2)$$

where $C_{ij}[k]$ represents the k th element of $\mathbf{C}_{ij}$. Then, (1) can be written as

$$b_q(i, j) = (y_q[0] - (2C_{ij}[0] - 1)a)^2 + (y_q[1] - (2C_{ij}[1] - 1)a)^2 + \dots + (y_q[n(K-1)-1] - (2C_{ij}[n(K-1)-1] - 1)a)^2. \quad (3)$$

This is equivalent to the Euclidean distance between the codeword symbols and the received symbols corresponding to the $(K-1)$ branches in the original low-connectivity trellis diagram. Similar to the low-connectivity trellis decoding, in the strongly connected trellis decoding, comparisons between the path metrics are required to determine the survivor path. Thus, the differences between the path metrics rather than the actual value of the path metrics are of concern. That is, one can arbitrarily add a constant to all the path metrics without changing the comparison result. This constant can be viewed as the one to be added to the composite branch metrics, since the path metrics are made up of the accumulated composite branch metrics. In (3), the received sequence $\mathbf{y}_q$ is used to generate the composite branch metrics of stage q so that each composite branch metric of stage q includes the constant $y_q[0]^2 + y_q[1]^2 + \dots + y_q[n(K-1)-1]^2$. Therefore, if this constant is subtracted from $b_q[i, j]$, the comparison result between the path metrics at stage q will not be changed. Furthermore, the expression $[(2C_{ij}[0] - 1)a]^2 + [(2C_{ij}[1] - 1)a]^2 + \dots + [(2C_{ij}[n(K-1)-1] - 1)a]^2$ is equal to $n(K-1)a^2$ whether $C_{ij}[k] = "0"$ or $"1"$, $k = 0, 1, \dots, n(K-1)$. Hence, the constant $n(K-1)a^2$ can be subtracted from $b_q[i, j]$ and the resulting value divided by $2a$ without altering the result of comparison between the path metrics of stage q . Therefore, (3) can be further modified as

$$\hat{b}_q[i, j] = -(2C_{ij}[0] - 1)y_q[0] - (2C_{ij}[1] - 1)y_q[1] - \dots - (2C_{ij}[n(K-1)-1] - 1)y_q[n(K-1)-1]. \quad (4)$$

For $K = 9$ and $r = 1/2$

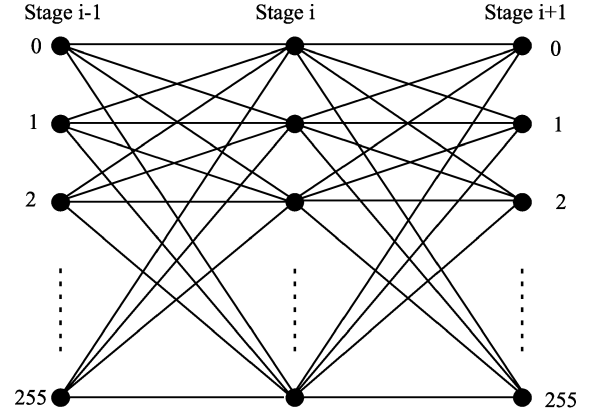
$$\hat{b}_q[i, j] = -(2C_{ij}[0] - 1)y_q[0] - (2C_{ij}[1] - 1)y_q[1] - \dots - (2C_{ij}[15] - 1)y_q[15]. \quad (5)$$

Thus, the computation of the metric for a composite branch is simplified into signed additions depending on whether $C_{ij}[k] = "0"$ or $"1"$, in the same way as presented in [6]. Note that this simplification could result in a negative value for the composite branch metric. As a result, the hardware implementation would become simple.

From (4), it is clear that before the composite branch metric $\hat{b}_q[i, j]$ is evaluated, the codeword sequence $\mathbf{C}_{ij}$ should be generated. As noted in [7], a convolutional encoder can be viewed as a linear time-invariant system in which the total response can be decomposed into two parts, the zero-input response and the zero-state response. Let N denote the total number of states, that is, $N = 2^{(K-1)}$. Thus, the codeword sequence $\mathbf{C}_{ij}$, with the initial encoder state i and the input sequence j , can be expressed as

$$\mathbf{C}_{ij} = \mathbf{C}_{i0} \oplus \mathbf{C}_{0j} \quad (6)$$

where $\mathbf{C}_{i0}$ represents all the codeword sequences corresponding to $i = 0, 1, \dots, (N-1)$ with zero input sequence,



0, 1, 2, ..., 255 represent the 256 states at a given stage

Fig. 1. Strongly connected trellis diagram for $K = 9$.

$\mathbf{C}_{0j}$ represents all the codeword sequences corresponding to $j = 0, 1, \dots, (N-1)$ with zero encoder state, and the symbol $\oplus$ represents bitwise modulo 2 addition. Obviously, if all the codeword sequences $\mathbf{C}_{ij}$ with $i, j = 0, 1, \dots, (N-1)$ are stored, it will need a huge memory space of $N^2n(K-1)$ bits. In view of (6), only the minimum sets of the codeword sequences $\mathbf{C}_{i0}$ and $\mathbf{C}_{0j}$ with $i, j = 0, 1, \dots, (N-1)$ are required to be known and stored, and the other codeword sequences can simply be derived from these $\mathbf{C}_{i0}$ and $\mathbf{C}_{0j}$.

B. Updating of Path Metrics

In the adaptive Viterbi algorithm of [12], instead of keeping all the possible distinct paths in each trellis stage, only a number of the most likely paths are kept and all the others discarded. As a result, only the survivor paths are involved in the evaluations in each trellis stage, and thus the amount of computations is reduced. The selection of the survivor paths at the trellis stage q is done by choosing, for each state, a path with the minimum metric and comparing it with $(d_m + T)$, where T is the discarding threshold value selected by the user, and d_m the minimum metric of all the survivor paths at the trellis stage $(q-1)$. Assuming a convolutional code with a constraint length of K and a code rate of $1/n$, in its strongly connected trellis diagram there will be 2^{K-1} number of states at each stage. Further, at each state of a stage, 2^{K-1} branches enter and the same number of branches leave. For the purpose of illustration, a strongly connected trellis diagram for $K = 9$ is shown in Fig. 1. For the q th strongly connected trellis stage, let $P_q(j)$ denote the survivor path metric and $Sur_q(j)$ the survivor state with the smallest path metric, where $j = 0, 1, \dots, 2^{K-1}-1$ denote the different states. Also, let d_m^q represent the minimum metric of all the survivor paths, and $S_q(j)$ the survivor information at the q th stage. The adaptive Viterbi algorithm can be formulated as follows.

At the start, the path metric, the survivor state, and the survivor information are initialized as

$$P_0(0) = 0 \quad (7)$$

$$Sur_0(0) = 0 \quad (8)$$

$$S_0(j) = \begin{cases} 1, & \text{for } j = 0 \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

Let $j_0, j_1, \dots, j_{L-1}$ represent the L survived states at the $(q-1)$ th stage. Then, $S_{q-1}(j)$ and d_m^{q-1} can be expressed as

$$S_{q-1} = \begin{cases} 1, & \text{for } j = j_0, j_1, \dots, j_{L-1} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

$$d_m^{q-1} = \min \{P_{q-1}(0), P_{q-1}(1), \dots, P_{q-1}(2^{K-1}-1)\}. \quad (11)$$

For $K = 9$

$$d_m^{q-1} = \min \{P_{q-1}(0), P_{q-1}(1), \dots, P_{q-1}(255)\}. \quad (12)$$

The survivor information $S_{q-1}(j)$ can be combined into the path metric value $P_{q-1}(j)$ and treated as a flag bit indicating the survivor of the j th state in the $(q-1)$ th stage. We can now find $P_q(j)$ and $Sur_q(j)$ as follows:

$$P_q(j) = \min \{P_{q-1}(j_0) + b_q(j_0, j), \dots, P_{q-1}(j_L) + b_q(j_L, j)\} \quad (13)$$

$$Sur_q(j) = \min^{-1} \{P_{q-1}(j_0) + b_q(j_0, j), \dots, P_{q-1}(j_L) + b_q(j_L, j)\} \quad (14)$$

where j represents the 2^{K-1} states of the q th stage, $\min$ represents the operation of taking the minimum metric of the survivor paths at state j (of stage q), and $\min^{-1}$ the operation of finding the state number in the $(q-1)$ th stage that yields the minimum metric of the survivor paths at state j of stage q .

In general, just as the Viterbi algorithm can be formulated as a matrix-vector computation [7], the path metric update here can be expressed as

$$\mathbf{P}_q = \mathbf{P}_{q-1} \otimes \mathbf{B}_q \quad (15)$$

where $\mathbf{P}_q$ represents the path metric row vector of the q th stage with 2^{K-1} elements, whose j th element is denoted by $P_q(j)$, $\mathbf{B}_q$ represents the $2^{K-1} \times 2^{K-1}$ adjacency matrix, whose ij th element is denoted by $b_q(i, j)$ representing the composite branch metrics from state i of stage $q-1$ to state j of stage q , and $\otimes$ represents the ACS operation according to the likelihood criterion.

It is clear that (13) can be viewed as an example for the path metric update at state j in (15), and it also shows that the ACS computations of the 2^{K-1} incoming paths at each state are performed selectively by the survivor information of the $(q-1)$ th stage. This means that only if the survivor information of the $(q-1)$ th stage is “1,” the ACS computations at q th stage are performed. The present survivor information $S_q(j)$ at each state can be determined by comparing $P_q(j)$ with $(d_m^{q-1} + T)$. If $P_q(j) < (d_m^{q-1} + T)$, then the q th stage survivor information $S_q(j)$ is set as “1,” otherwise as “0.” Since the Euclidean distance for computing a composite branch metric has been modified into signed additions in the same way as presented in [6], the composite branch metrics and hence, the path metrics could have negative values. Consequently, the minimum metric d_m at a given stage could also be negative. In this case, the most optimum value for the threshold value T should be negative.

The selection of the survivor paths and the survivor paths update can be repeated by using the procedure described above at

each strongly connected trellis stage. Finally, the decoded data can be obtained through a traceback of the survivor path.

C. Modulo Arithmetic for ACS

According to (13), the recursive path metric update at each state of a stage results in an unbounded word growth due to the accumulations by the composite branch metric of the stage. To avoid normalization, which costs additional circuits in hardware implementation and increases the processing time, the modulo-arithmetic approach [15], [16] is adopted here. It is known that for the original radix-2 trellis, the Viterbi algorithm inherently bounds the maximum dynamic range $\Delta_{\max}$ of the path metric at each state as

$$\Delta_{\max} \leq \lambda_{\max} \log_2 N \quad (16)$$

where N is the number of states and $\lambda_{\max}$ is the maximum branch metric. In the Viterbi decoding process, any two updated path metrics at a state of stage q , $P_q[i, j]$ and $P_q[l, j]$, are compared using subtraction. It has been shown in [9], [10] that if $|P_q[i, j] - P_q[l, j]| < \Delta_{\max}$, the comparison can be evaluated as $(P_q[i, j] - P_q[l, j]) \bmod 2\Delta_{\max}$ without any ambiguity. Hence, the updated path metrics at any state of a stage can be computed modulo $2\Delta_{\max}$. In this case, the wordlength of the path metric can be evaluated by

$$\Gamma_{\text{bit}} = \log_2 \Delta_{\max} + 1. \quad (17)$$

The strongly connected trellis can be viewed as a radix- $2^{(K-1)}$ trellis obtained from the original radix-2 low-connectivity trellis. In the strongly connected trellis decoding, the dynamic range of the path metric increases, since a $(K-1)$ -stage composite branch metric is added at each strongly connected stage. The wordlength of the strongly connected path metric can be deduced as

$$\Gamma_{\text{bit}} = \log_2 (\Delta_{\max} + (K-1)\lambda_{\max}) + 1. \quad (18)$$

For $K = 9$ and $r = 1/2$, if three-bit soft demodulation decision is used, the value of $\lambda_{\max}$ is 14, and hence, the value of $\Delta_{\max}$ is 112. Accordingly, the word length of Γ_{bit} is equal to 10 bits including a sign bit.

D. Radix- 2^{K-1} Trellis Traceback Update

The traceback update can be viewed as a process to generate decoded outputs through tracing back the survivor decisions made according to the comparison results of the path metrics of each state at a stage, and stored along the trellis stages. The radix- $2^{(K-1)}$ trellis traceback update can be deduced starting from the traceback update with the original radix-2 trellis [16].

Given a radix-2 trellis, let n represent the traceback starting stage, S_n an arbitrary state of stage n , and d_n^S the survivor decision of state S_n . The state of the previous stage S_{n-1} from which the survived path at state S_n results is given by

$$S_{n-1} = d_n^S(S_n \gg 1) \quad (19)$$

where $S_n \gg 1$ represents the right shift of the constant S_n by one bit. Hence, the state S_{n-1} can be traced back by right shifting out the state S_n one bit at one end of the state register. Meanwhile, the decision d_n^S is shifted in at the other end. If two

trellis stages are merged together, an equivalent radix-4 trellis is derived. In such a case, the traceback iterations from the n th stage to the $(n-2)$ th stage can be expressed as

$$\begin{aligned} S_{n-2} &= d_{n-1}^S(S_{n-1} \gg 1) \\ &= d_{n-1}^S(d_n^S(S_n \gg 1) \gg 1) \\ &= d_{n,n-1}^S(S_n \gg 2) \end{aligned} \quad (20)$$

where $d_{n,n-1}$ is the composite 2-bit radix-4 decision. Similarly, if $(K-1)$ trellis stages are merged together, the strongly connected radix- $2^{(K-1)}$ trellis is obtained. In this case, the traceback iterations from the n th to $(n-(K-1))$ th stage can be expressed as

$$S_{n-(K-1)} = d_{n,n-1,\dots,n-(K-2)}(S_n \gg (K-1)) \quad (21)$$

where $d_{n,n-1,\dots,n-(K-2)}$ is the composite $(K-1)$ -bit radix- $2^{(K-1)}$ decision.

For $K=9$ and $r=1/2$,

$$S_{n-8} = d_{n,n-1,\dots,n-7}(S_n \gg 8). \quad (22)$$

Obviously, instead of decoding only one bit per traceback iteration in the original radix-2 low-connectivity trellis Viterbi decoding, $(K-1)$ bits (8 bits for $K=9$ and $r=1/2$) can be decoded per traceback iteration in the traceback decoding process based on the strongly connected radix- $2^{(K-1)}$ trellis. Thus, the strongly connected trellis decoding can increase the throughput by a factor of $(K-1)$ compared to that of using the original low connectivity trellis decoding. Since any state of the radix- $2^{(K-1)}$ trellis diagram can be represented by $(K-1)$ binary bits, (21) shows that the bits representing a state of the $[n-(K-1)]$ th stage is equivalent to the $(K-1)$ -bit $d_{n,n-1,\dots,n-(K-2)}$ decision.

III. SIMULATION RESULTS

The error performance and the average number of computations per stage for the proposed adaptive as well as the nonadaptive Viterbi algorithm based on the strongly connected trellis decoding in the presence of additive white Gaussian noise is simulated using the C-language. This simulation study is carried out for decoding a convolutional code with a constraint length $K=9$ and a code rate of $1/2$, in which a BPSK modulation scheme is used. In the simulations, one million random data bits are generated and encoded. Then the encoded data bits are converted into symbol values and the additive white noise is imposed on the symbol values. Subsequently, the symbol values with additive white noise is quantized with 3 bits, and these quantized data are used as inputs for decoding by the nonadaptive and adaptive Viterbi algorithms.

Fig. 2 shows the error performance for both the convolutional-coded binary data using the proposed algorithm and the uncoded binary data. The error performance is expressed by the bit error rate (BER) as a function of E_b/N_o , where E_b denotes the signal energy and N_o the noise density. It can be seen from this figure that the proposed adaptive Viterbi algorithm based on the strongly connected trellis decoding can provide at least a 5-dB coding gain at a bit-error-rate of 10^{-5} , when the threshold value T is chosen to be greater than -8 . Fig. 3

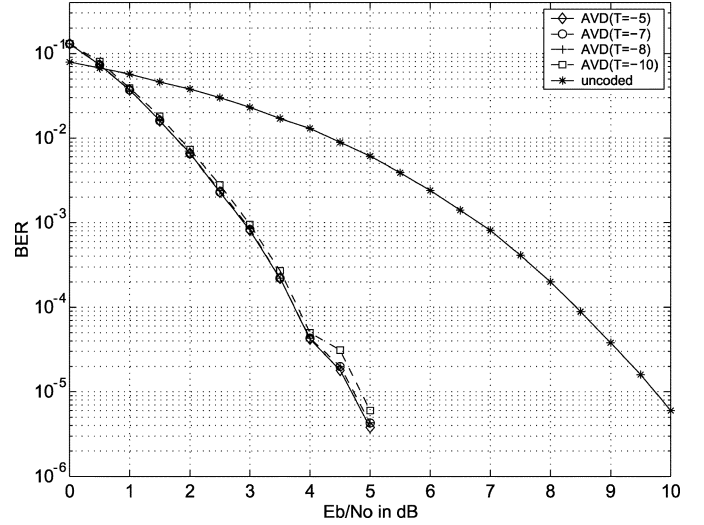


Fig. 2. Error performance for coded and uncoded binary data.

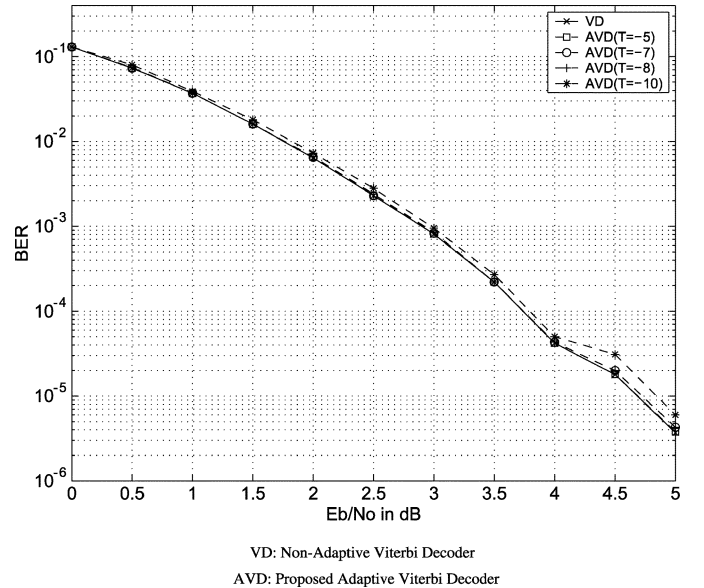


Fig. 3. BER performance as a function of E_b/N_o .

shows the error performance of the proposed algorithm for four different values of T as well as that of the nonadaptive Viterbi algorithm. When the threshold value T is chosen to be -5 , the error performance curve for the proposed adaptive Viterbi algorithm completely coincides with that of the nonadaptive one, whereas when $T=-7$ and $T=-8$, there is a slight degradation in the performance for E_b/N_o greater than 4 dB. However, for $T=-10$, this degradation is not negligible. Thus, the error performance can be brought closer to that of the nonadaptive Viterbi algorithm by increasing the threshold value.

The computational complexity expressed by the average number of ACS computations per stage as a function of E_b/N_o is shown in Fig. 4 for the nonadaptive Viterbi algorithm as well as for the proposed algorithm for four values of T . The average amount of ACS computations per stage for the adaptive Viterbi algorithm is the mean taken over a large number of iterations (one million/ $(K-1)$). For the nonadaptive Viterbi algorithm, this is a fixed number. It is seen that for a coding gain

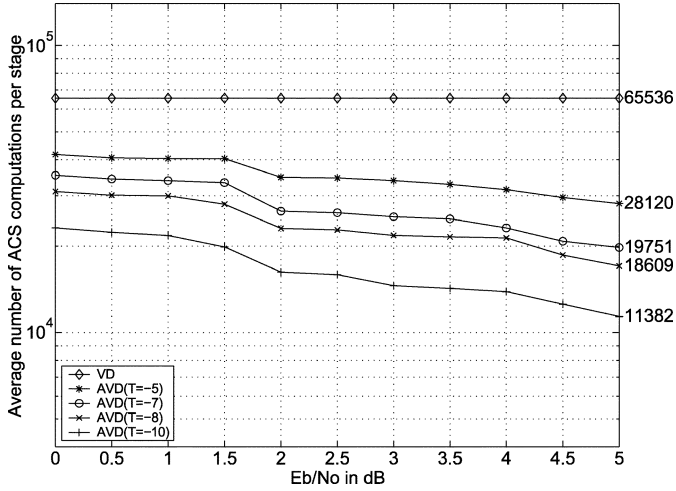


Fig. 4. Average number of ACS computations per stage as a function of E_b/N_o .

of 5 dB, the average number of ACS computations per stage using the proposed algorithm with $T = -5$ is 42.8% of that of the nonadaptive Viterbi algorithm for a strongly connected trellis decoding. This number reduces to 30.1%, 28.4%, and 17.4%, respectively, for $T = -7$, $T = -8$, and $T = -10$. However, as pointed out earlier, there is a degradation in the error performance for $T = -10$.

Thus, it is seen that by using the proposed algorithm with a threshold value of -7 , one can achieve a computational complexity of only 30% of that of the nonadaptive algorithm, while maintaining virtually the same error performance.

In our simulations, the computational precision is selected to conform to the requirements of the hardware computing units. The branch metrics $\hat{b}_q[i, j]$ are computed using (5) that will actually be used in the hardware implementation. In addition, the traceback length is chosen to be 8 strongly connected trellis stages, the same number as the one used later in our hardware implementation. According to (22), for each strongly connected trellis decoding process, there are 8-bit binary data to be decoded in this design. Thus, the traceback length of eight strongly connected trellis stages is equivalent to a traceback length of $8(K - 1) = 64$ low-connectivity trellis stages. It is known that in the Viterbi algorithm, the traceback length should be at least $5K$ in order to have a high probability for all the survived paths from all the states to converge to one path from which the decoded data is retrieved [1]. Thus, the traceback length used here is large enough so as not to affect the error results. Therefore, the simulation results for the error performance and average number of ACS computations per stage are indicative of the results for the hardware implementation.

IV. DESIGN AND IMPLEMENTATION OF MODIFIED ADAPTIVE VITERBI DECODER

In order to design and implement the modified adaptive Viterbi decoder, we choose the following specifications that have been used in [3] for hardware implementation of the Viterbi decoder:

- 1) constraint length: $K = 9$ (256 states);
- 2) code rate: $r = 1/2$;

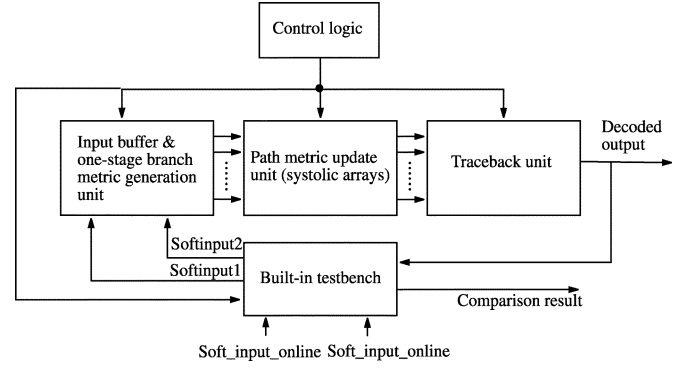


Fig. 5. Block diagram of the adaptive Viterbi decoder.

- 3) generator polynomials: $g(1) = 561$ and $g(2) = 743$;
- 4) survivor path length: $L = 64$;
- 5) decision level: 3-bit soft decision.

The block diagram of the modified adaptive Viterbi decoder is shown in Fig. 5. In this decoder, the input buffer and one-stage branch metric generation unit is designed to collect the two soft input sequences and to compute all the possible branch metrics corresponding to the eight low-connectivity trellis stages. The path metric update unit is designed to generate the composite branch metrics and to process the matrix-vector ACS computation. The traceback unit can retrieve the decoded sequence from the survivor path memory through a traceback strategy. The built-in testbench provides a functional testability to the design, and the control logic provides the timings for all the input signals to each unit. We now consider the design of each of the units shown in Fig. 5.

A. Systolic Array Architecture-Based Branch and Path Metrics Update

The modified adaptive Viterbi algorithm can be implemented using the systolic array architecture shown in Fig. 6, in which time multiplexing, arithmetic pipelining, and spurious toggle reduction techniques are exploited. Further, a scheme providing a tolerance to clock-to-data skews is also investigated.

1) *Adjacency Matrix Partitioning and Time Multiplexing*: For $K = 9$, the matrix-vector ACS computation as given in (15) can be processed by the 256×256 adjacency matrix $\mathbf{B}_q$ and the 1×256 path metric vector $\mathbf{P}_{q-1}$. In this design, the adjacency matrix $\mathbf{B}_q$ is partitioned into four 256×64 submatrices, and the four submatrices along with the path metric vector $\mathbf{P}_{q-1}$ are used in (15) to update the corresponding $256 (= 4 \times 64)$ path metrics of a given stage. This is achieved by the four pairs of interconnected linear systolic arrays shown in Fig. 6. Each pair of the linear systolic arrays is formed as suggested in [7] and is shown in Fig. 7, where the top interconnected four processors represent the systolic array BM₄ and the bottom interconnected four processors the systolic array ACS₄ of Fig. 6. In the top systolic array, the composite branch metric $\hat{b}_q[i, j]$ is generated by using (5) and (6). The codeword $\mathbf{C}_{0j}$ stays inside each processor, and $\mathbf{C}_{i0}$ moves to the right. In the bottom systolic array, the path metric $P_q(j)$ of stage q is computed according to (13) and decisions on the survivor paths are made according to (14). The intermediate ACS result denoted by $R_q(j)$ stays inside the j th processor, whereas

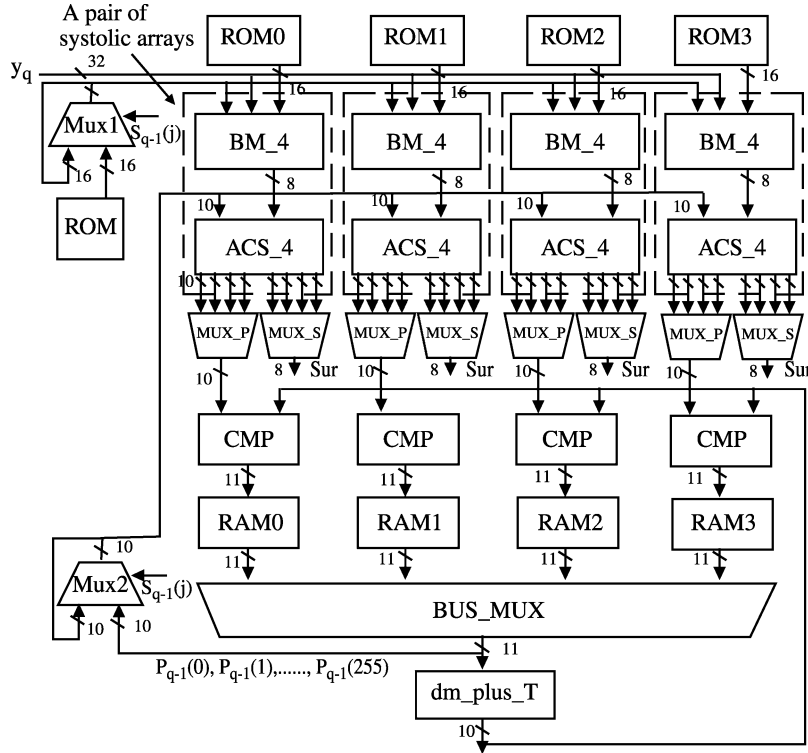


Fig. 6. Systolic array-based architecture for the adaptive Viterbi decoder.

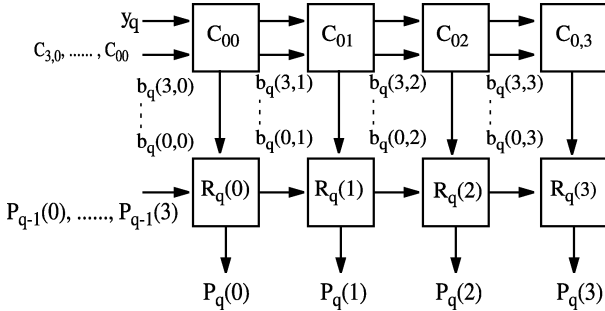


Fig. 7. Architecture of a pair of systolic arrays.

$P_{q-1}(j)$ of stage $q-1$ moves to the right. All data movements in the two systolic arrays are synchronized. This design can thus be viewed as a “result stay” systolic array architecture. As shown in Fig. 6, for computing the corresponding four subsets of $\mathbf{B}_q$ at each stage, the codeword vector $\mathbf{C}_{0j}$ is divided into four 1×64 subvectors to be stored in four ROMs, ROM0 to ROM3, and the codeword vector $\mathbf{C}_{i0}$ stored in the ROM. Globally, the four submatrix-vector ACS computations are carried out simultaneously by the corresponding four pairs of systolic arrays. Locally, inside each pair of the systolic arrays, the corresponding submatrix-vector ACS computation is time multiplexed. For a given K , the number of trellis states at a given stage is equal to 2^{K-1} . If the time-multiplexing technique is not employed, the number of pairs of systolic arrays (each array having 4 processors) is $(2^{K-1})/4 = 2^{K-3}$. For $K = 9$, this is 64, and the total number of array processors is 512. As K increases, the number of array processors will increase exponentially, which is the same as in the case of the conventional

butterfly architecture-based parallel Viterbi decoder. However, we employ a time-multiplexing technique. In this technique, when $K = 9$, each pair of systolic arrays (each array having four processors) processes 64 out of 256 states sequentially in 16 iterations. For a given K , the more the number of systolic array pairs used, the fewer the number of iterations needed. Thus, as K increases, our time-multiplexing approach provides the flexibility to trade off the number of iterations with the number of systolic array pairs.

2) *Survivor Information Generation and Spurious Toggle Reduction:* In the four CMP units in Fig. 6, the survivor information corresponding to the updated path metrics from the four pairs of systolic arrays are obtained by comparing $P_q(j)$ with $(d_m^{q-1} + T)$. In the dm_plus_T unit, d_m^{q-1} is obtained by taking the minimal value among the path metrics at the $(q-1)$ th stage, and $(d_m^{q-1} + T)$ is calculated. The updated path metric subvectors from the four pairs of systolic arrays are selected for the four CMP units by the four multiplexers (MUX_P), and the path metric subvectors together with the survivor information are written into and read out from the four RAMs, RAM0 to RAM3. The two multiplexers, Mux1 and Mux2, are used to keep the input data streams to the pairs of systolic arrays unchanged from their previous values, whenever the corresponding path metric to be computed does not survive, so that there is no switching activity in the arithmetic units of the systolic arrays. This process of eliminating the unnecessary switching is called spurious toggle reduction.

3) *Arithmetic-Pipelining Processors:* It is seen from (5) that to generate a composite branch metric, one needs to process 15 signed additions, whereas to compute a path metric only three ACS operations are needed. Usually, the global clock period has

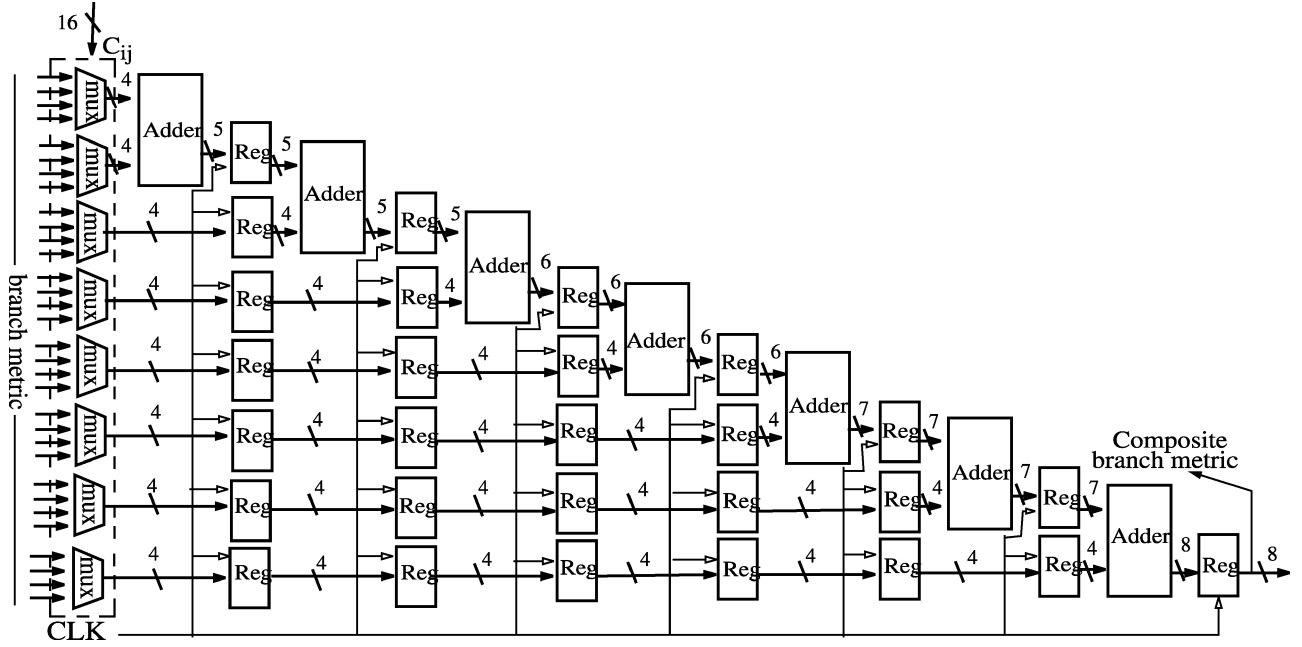


Fig. 8. Arithmetic pipelining processor for BM_4.

to be the higher of these two computational times, plus some safety margin so as to achieve the synchronization of the whole system. This is obviously undesirable from the point of view of speed. Fortunately, the use of a smaller clock period could be a solution not only to allow different operations in the array network to take different time periods, but also to speed up the design. Since one composite branch metric computation and one path metric computation consist of a number of arithmetic steps, the processors corresponding to the two types of computation can be designed with arithmetic-pipelining in which each arithmetic step is treated as a pipeline stage by adding appropriate registers. This implies that each pair of systolic arrays can be realized with a two-level pipelining, pipelining at the global array level from processor to processor and pipelining within a processor.

The arithmetic-pipelining processor for a BM_4 is shown in Fig. 8. A composite branch metric can be viewed as the summation of the $K - 1 = 8$ corresponding one-stage branch metrics in the original low-connectivity trellis decoding. Therefore, these one-stage branch metrics can be generated before the corresponding composite branch metric is computed. In this design, the computation involved in (5) is divided into two parts, the computation of the eight one-stage branch metrics and the computation of the summation of these one-stage branch metrics. A module placed outside the arrays is designed to generate, by using a two-bit codeword, all the possible values for each of the eight one-stage branch metrics corresponding to a composite branch metric at a given stage. At a given stage, the array processors in a BM_4 first select, according to the codeword sequence, the eight computed one-stage branch metrics, and then perform seven signed additions to obtain the corresponding composite branch metric. Thus, the word length corresponding to the maximum composite branch metric can be 7 bits plus a sign bit. In this way, an array processor for a BM_4 can be less complex compared to an array processor with 15 signed additions imple-

mented by 15 pipeline stages. As a result, in an FPGA implementation, the resource utilization can be reduced.

Path metric update processes the ACS computations with the composite branch metrics of the current stage and the path metrics of the previous stage. In general, a feedback is needed in path metric updating, since the path metrics of the previous stage should be fed back to the input of an ACS processor for updating the path metric at the current stage. In the butterfly architecture-based Viterbi decoders [3]–[6], ACS computations cannot be pipelined because of the feedback that would occur between the pipeline stages. However, in the proposed systolic array-based architecture, the ACS computations can be carried out in pipeline. In this design, each path metric update of the q th stage processes 256 ACS computations with 256 corresponding composite branch metrics of the q th stage and 256 path metrics of the $(q - 1)$ th stage. In view of this large number of inputs to be processed at a given stage, the proposed architecture is very suitable. Furthermore, the path metrics of the q th stage are fed back to the input of the corresponding processor at the same time as the corresponding composite branch metrics of the $(q + 1)$ th stage are generated. More importantly, since a latency of outcome exits in a pipelined design, during the sixteenth (final) iteration while updating the path metrics for the q th stage, the updating of the path metrics for the $(q + 1)$ th stage has already commenced. However, a two-level pipelining can ensure that there is no conflict between the updations of the path metrics for the q th and the $(q + 1)$ th stages.

The arithmetic-pipelining processor for an ACS_4 is shown in Fig. 9. This processor is implemented with two pipeline stages. The adder in the first pipeline stage is used to compute the 256 path metrics at any of the corresponding 16 states of a given stage, whereas the adder in the second pipeline stage is used to make comparisons amongst the 256 path metrics computed by the first pipeline stage. According to (16) and (18), the length of the adders including a sign bit in the two

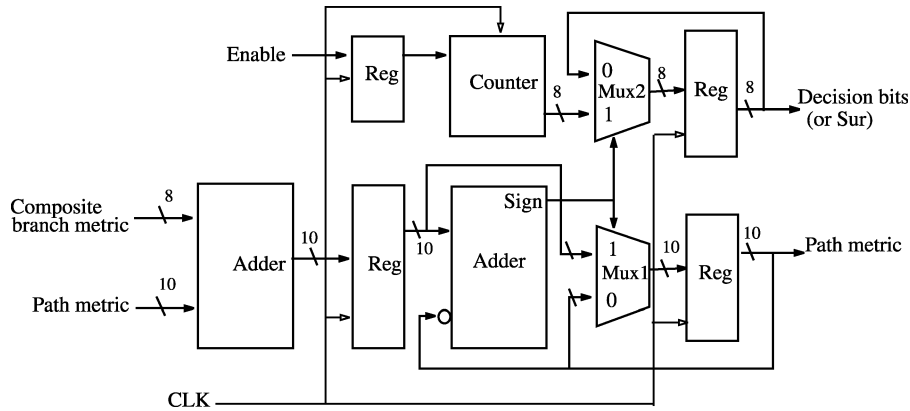


Fig. 9. Arithmetic-pipelining processor for ACS-4.

pipeline stages should be 10 bits in order to perform the modulo arithmetic. In the second pipeline stage, the comparison between the path metrics computed at the current and previous clock cycles at a state of a given stage are carried out by adding the path metric at the current clock cycle to the compliment of the path metric at the previous clock cycle, increased by unity. In this way, as presented in [15], the sign-bit of the adder can represent the result of the comparison between the two path metrics. If the sign-bit is “1,” the path metric at the current clock cycle is smaller than the path metric at the previous clock cycle, and if the sign-bit is “0,” then the former is larger than the latter. In the meantime, the path metric with the smaller metric at the state of a given stage is selected by the multiplexer (Mux1) and sent to the Register. In addition, the 8-bit counter in the second pipeline stage is used to represent the 256 possible states of the previous stage from which the paths originate and terminate at the various states of the present stage. One of these 256 paths would be the survivor path depending on the sign bit of the adder in the second pipeline stage, and is selected by the multiplexer (Mux2) at each clock cycle. Thus, at a given stage, the path metric and the survivor corresponding to any of the 16 states can be updated through 256 clock cycles. It should be mentioned that the critical path of the design as determined from the Xilinx report is the second pipeline stage and consists of the 10-bit adder and the multiplexer (see Fig. 9).

The synchronization between ACS₄ and BM₄ can be achieved by simply designing a timing scheme that makes them start at different time instants. This means that at a given stage the processors in an ACS₄ will not start their computations until the results from the processors in the corresponding BM₄ have been generated. In this way, throughput of the design will not be changed, but a latency between ACS₄ and BM₄ is introduced. This is in fact the latency of any processor in BM₄.

4) Clock-To-Data Skew and a Scheme for Its Avoidance: The systolic array architecture described above provides a highly synchronized Viterbi decoder in which a source-synchronous clocking scheme is used, and the clock frequency determines the rate of the decoding. Although the source-synchronous clocking scheme can achieve signal integrity in the design, clock-to-data skew remains a problem to be solved. In an FPGA implementation-based design, the clock-to-data skew is caused by the threshold and delay mismatch of the driver

output cells, edge-rate mismatch between the clock and the data-output cells, and trace length mismatch between the clock and the data paths. In this design, a scheme is proposed to avoid large clock-to-data skews and to provide a tolerance for small clock-to-data skews.

Because of the use of four pairs of systolic arrays each having four processors, instead of a pair of systolic arrays each having 16 processors, to process the large matrix-vector ACS computations, large clock-to-data skews have been avoided. Clock-to-data skews caused by delay mismatch between clock and data increases with the size of a systolic array. The larger the systolic array, the more the pipeline stages involved, and thus the larger the clock-to-data skews generated. Therefore, the clock-to-data skews caused by a delay mismatch between the clock and the data in a 16-processor systolic array can be larger than in a 4-processor systolic array. Thus, the use of four pairs of systolic arrays, each having 4 processors, is appropriate for limiting the clock-to-data skews.

A scheme for avoiding large clock-to-data skews is devised by using 4-input look-up tables (LUTs) provided in the FPGA device to implement the array processor for a BM₄, shown in Fig. 8. In this array processor, there are seven pipeline stages, and each of the eight one-stage branch metrics is transferred and computed through the various pipeline stages. If the registers of this array processor are implemented by using the conventional D flip-flops (DFFs) provided in the XC2V1000-4FG256 chip, a trace length from one DFF to another can be different depending on the placement and routing procedure in the FPGA implementation. As a result, there can be large clock-to-data skews caused by the trace length mismatches between the clock and the data paths in the design. However, the Xilinx Virtex-II XC2V1000-4FG256 chip provides a large number of 4-input LUT, that can be used to build a variable-tap shift register. Once the LUT-based variable-tap shift register is used to implement the registers in the array processor, a uniform trace length from register to register through the pipeline stages can be attained. Thus, large clock-to-data skews caused by the trace length mismatches between the clock and the data paths can be avoided.

On the other hand, since clock-to-data skews are inevitable in synchronous designs, there should be a scheme to provide a tolerance to the clock-to-data skews so that the timing violations caused by the clock-to-data skews can be avoided. Simu-

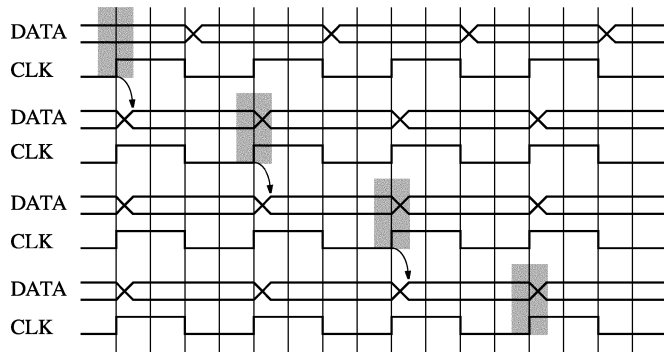


Fig. 10. Timing diagram without clock-to-data skew tolerance.

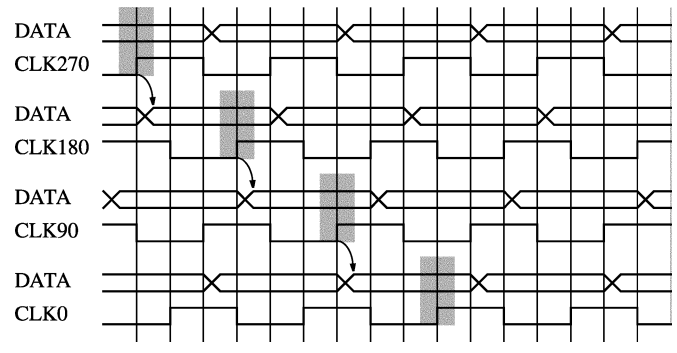


Fig. 12. Timing diagram with clock-to-data skew tolerance.

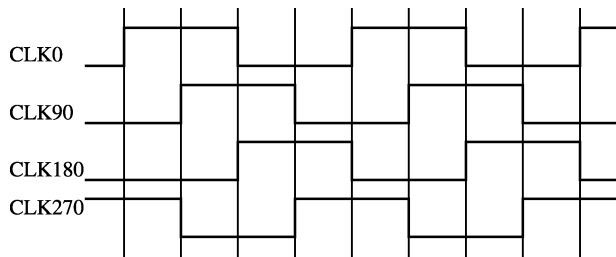


Fig. 11. Characteristics of the clock outputs.

lation studies have shown that once the design is implemented, the trace length mismatch between the clock and the data paths is a crucial factor to cause clock-to-data skews. If each pair of the systolic arrays is triggered by a unique clock, the design for the four synchronized pipeline stages will not be tolerant to clock-to-data skews, as shown in Fig. 10. The arrows in the figure illustrate that the data is transferred from one pipeline stage to the next at every rising edge of the clock, and the shaded areas show that the rising edge of the clock is very close to the transition phase of the data sequence. As a result, once there are trace length mismatches in the implementation, the setup time or hold time of the DFFs used in the pipeline register can be violated by the clock-to-data skews, and the outputs corresponding to these mismatches will be indeterminate.

Fortunately, the digital clock manager (DCM) in the Virtex-II XC2V1000 provides multiple phases of the source clock, CLK0, CLK90, CLK180, and CLK270, which make it possible to exploit a timing scheme with four phase-shifted clocks to provide a tolerance to clock-to-data skews in the design, as shown in Fig. 11. The timing diagram with the four-phase shifted clocks is shown in Fig. 12. CLK270 with the largest phase lag from the source clock is used to drive the first pipeline stage, and CLK0 with zero phase lag is used to drive the last pipeline stage. Likewise, CLK90 is used to drive the second pipeline stage and CLK180 is used to drive the third pipeline stage. Clearly, the four phase-shifted clocks can provide a 25% clock period safety margin for the data to be stable so that it can be retrieved at each pipeline stage when the clock is delayed, whereas they can provide a clock period safety margin of almost 75% when the data is delayed. Thus, the four phase-shifted clocks can compensate the clock-to-data skews at each pipeline stage, if a clock-to-data skew appears.

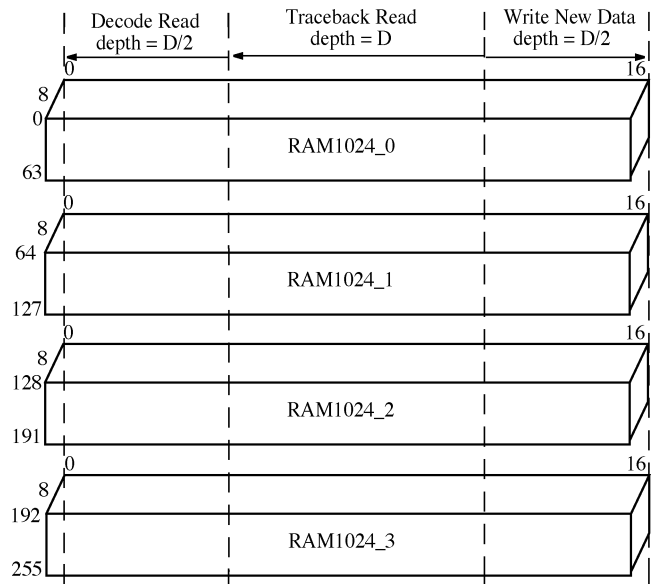


Fig. 13. Survivor path memory structure.

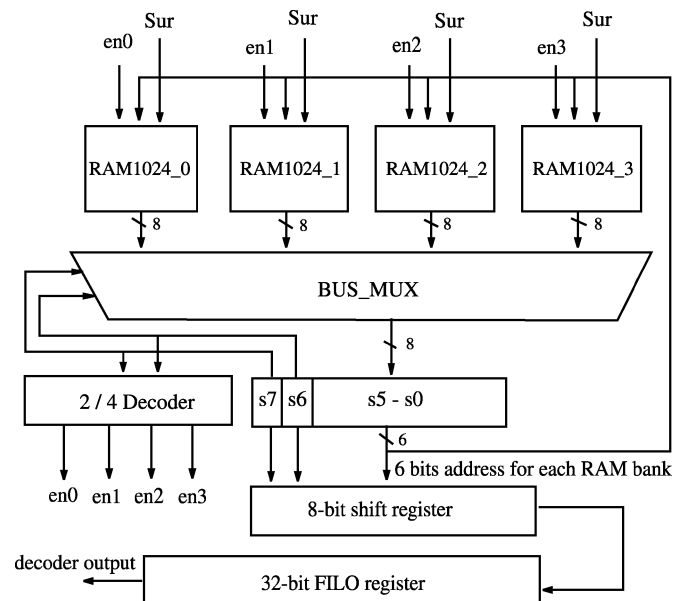


Fig. 14. Architecture of the traceback unit.

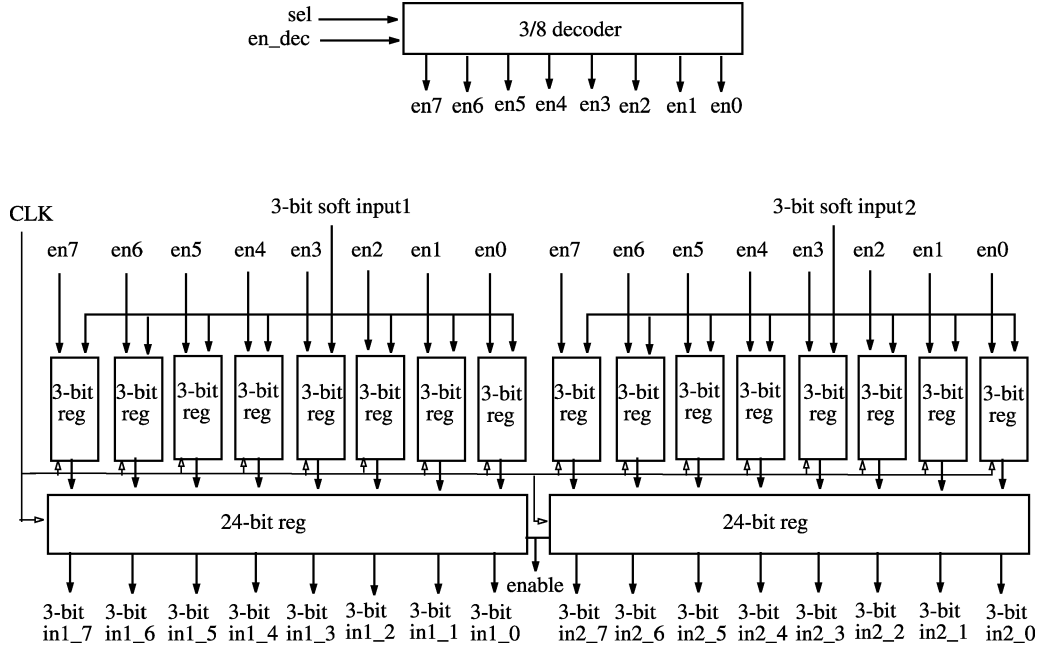


Fig. 15. Input buffer for the two soft inputs.

B. Traceback Strategy and Traceback Unit

In the FPGA-based implementation, the memory for storing the survivor paths is divided into four memory banks, each of which stores 64 survivor paths. The traceback length of the design is chosen as $D = 8$ that is equivalent to a traceback length of $8(K - 1) = 64$ low-connectivity trellis stages. As mentioned earlier in Section III, this is large enough to ensure the convergence of the survivor paths. Since 2-D circular memory addressing scheme [6] is used in this design, the size of each memory bank should be equal to $64 \times 2-D = 1024$, and its wordlength 8 bits. The survivor path memory is organized as shown in Fig. 13. There are three operation phases in the management of the memory, write new data (WR), traceback read (TB), and decode read (DR), which occupy memory with depths of $D/2$, D , and $D/2$, respectively. These three operations access the three logical blocks of the memory, respectively, by using 2-D circular memory addressing scheme [6], in which the traceback read and the decode read operations are performed by using one pointer instead of multiple ones. In view of the memory depths of WR, TB, and DR, it is noted that the read rate should be three times that of the write rate, so that the two read-phases and one write-phase can be performed simultaneously within the same time period. The write pointer advances forward from stage to stage in the trellis, and the decisions made by the ACS computations at all the trellis states regarding the survivors are written into the memory block that is just freed by the decode read operation. The read pointer traces back from stage to stage so that the retrieved data after each read operation can be treated as a pointer to indicate the corresponding state number of the previous stage. When the decisions are written into the memory for one stage, the read pointer will traceback the memory by three stages. After D such tracebacks, all the survivor paths will converge and the actual decoding takes place. There are $D/2 \times 8 = 32$ bits of data decoded at each trace-

back iteration, and these retrieved data should be rearranged in its original order.

The architecture of the traceback unit is shown in Fig. 14. The four block select RAMs each of size of 1024×8 bits provided by the FPGA chip are used to implement the four memory banks, RAM1024_0 to RAM1024_3. At each stage, the survivor decisions made by the four pairs of the systolic arrays in Fig. 6 are written into the four corresponding memory banks. At each traceback read, the previously retrieved data (the state number) points to the current data to be read. The most significant two bits s_7 and s_6 , of the previously retrieved data in the register are sent to the 2/4 Decoder and BUS_MUX. These bits are used to select the memory bank to be accessed and to have its stored data retrieved. The least significant 6 bits, s_5 to s_0 , of the previously retrieved data in the register are used to address the 64 decisions in one of the memory banks selected at each read operation. In other words, at every traceback operation, one of the 256 decisions (corresponding to the 256 survivor paths) stored in the four memory banks can be read by using the previously retrieved data as a pointer. After D tracebacks, corresponding to the eight strongly connected trellis stages, the decoded data is loaded into the 8-bit shift register and then shifted to the first-in-last-out (FILO) register. Finally, the decoded data is sent out in the reverse order from the FILO register.

C. Input Buffer and One-Stage Branch Metric Generation

The input buffer is designed to gather the two soft input sequences corresponding to a given strongly connected trellis stage so that the composite branch metrics of the stage can be generated. As shown in Fig. 15, the two soft inputs are sent to the corresponding 3-bit registers depending on the enable signal generated by the 3/8 decoder. The 3/8 decoder outputs the eight enable signals in a circular way so that the two soft input sequences corresponding to a strongly connected trellis stage are loaded into the 3-bit registers. The sequences are

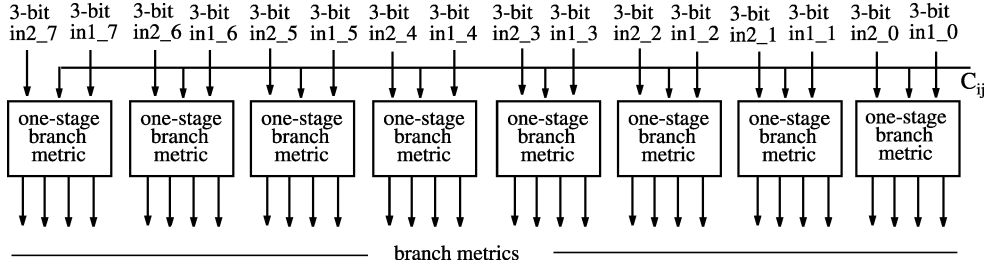


Fig. 16. Unit to compute the eight one-stage branch metrics corresponding to a composite branch metric.

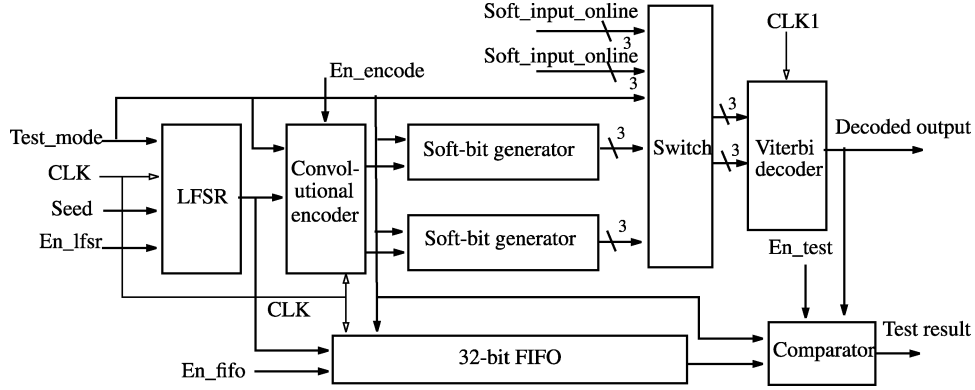


Fig. 17. Architecture of the built-in testbench.

TABLE I
ONE-STAGE BRANCH METRIC EVALUATION

Code Word C_{ij}	One-Stage Branch Metric
0 0	$in1 + in2$
0 1	$in1 - in2$
1 0	$-in1 + in2$
1 1	$-in1 - in2$

then buffered into two 24-bit registers and used to compute the corresponding eight one-stage branch metrics. Each of the eight one-stage branch metrics has four possible results depending on the corresponding two-bit codeword in the original low connectivity trellis stage. Table I gives the expressions for the evaluation of a one-stage branch metric under the four possible values of the two-bit codeword, where $in1$ and $in2$ represent the two soft inputs at the low-connectivity trellis stage. The eight one-stage branch metrics corresponding to a composite branch metric are computed by the unit shown in Fig. 16, in which each one-stage branch metric is computed under the four possible values of the two-bit codeword, and is output to the array processors in BM_4s shown in Fig. 8.

D. Testability of the Design

In order to test the functionality of the Viterbi decoder offline, a testbench is built. The testbench along with the Viterbi decoder to be tested is shown in Fig. 17. The linear feedback shift register (LFSR) is used to generate a 32-bit random input sequence. The convolutional encoder is used to encode the random input sequence generated by the LFSR. The two soft-bit generators are used to convert the two codeword sequences generated by the encoder into two 3-bit soft input sequences to be

selected by the switch as the inputs to the Viterbi decoder. The 32-bit first-in first-out (FIFO) register is used to buffer the 32-bit sequence generated by the LFSR so that they can be used to make comparisons with the corresponding decoded 32 bits. In this way, the functionality of the Viterbi decoder can be tested. The system can operate in either the test mode or the on-line mode controlled by the signal “test_mode.”

V. FPGA IMPLEMENTATION AND COMPARISON

In order to study the advantage of the proposed adaptive decoder in terms of power consumption, the proposed decoder as well as a nonadaptive Viterbi decoder based on strongly connected trellis decoding are implemented using a systolic array architecture. The difference between the implementations of the nonadaptive Viterbi and the proposed algorithms is only in the path metric update units, all other parts of the design being identical. The path metric update unit for the Viterbi algorithm is shown in Fig. 18. This unit is the same as the one shown in Fig. 6 for the proposed adaptive decoder except that the former unit does not include the four CMPs, the two multiplexers, and the dm_plus_T block, which are needed for the latter unit for generating the survivor information and for eliminating the spurious toggles. In this paper, the designs for both the nonadaptive and the adaptive Viterbi decoders are modeled using VHDL, simulated by Synopsys VSS, synthesized by Synopsys Design Compiler and post synthesized by Xilinx Design Manager target for Virtex-II XC2V1000-4FG256 FPGA logic. The design simulations have been done based on both the VHDL codes at RTL and the VHDL codes generated by Xilinx design manager after post synthesis. Further, the two decoders have been implemented using a Virtex-II FPGA prototype board for XC2V1000-

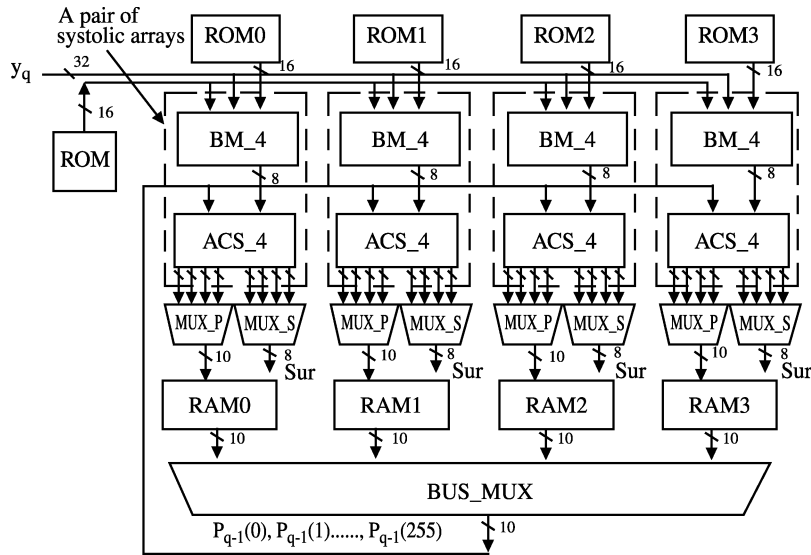


Fig. 18. Systolic array-based architecture for the nonadaptive Viterbi decoder.

TABLE II
POWER AND AVERAGE TOGGLE RATES IN CLB LOGIC FOR THE VITERBI DECODER AND THE ADAPTIVE VITERBI DECODER

Figures	Module	Non-Adaptive Viterbi Decoder		Adaptive Viterbi Decoder	
		Average Toggle Rate %	P_{int} Subtotal (mW)	Average Toggle Rate %	P_{int} Subtotal (mW)
Figs. 6, 18 (ACS ₄)	acsPEx16	50%	114	15%	40
	acsCNTx16	25%	6	25%	6
Figs. 6, 18	MUX _P s & MUX _S s	2%	0	2%	0
Fig. 6	cmpPEx4	X	X	2%	1
	dm_plus_T	X	X	50%	4
	Mux1 & Mux2	X	X	15%	0
Figs. 6, 18 (BM ₄)	brcPEx16	50%	417	15%	138
	brcREGx16	55%	9	17%	3
Fig. 15	recBUF	12%	16	12%	16
Fig. 14	decOUT	12%	1	12%	1
Fig. 17	testPKG	50%	12	50%	12
Control Logic (No Figure)	conLOGIC	50%	51	50%	51
Total			626	Total	272

4FG256 and tested by the built-in testbench for their functionality.

There are two types of power consumption in FPGAs, namely, static and dynamic. In Virtex-II, the leakage current is the only source of static power dissipation. This static power is between 5%–20% of the total power dissipation in Virtex-II, depending on such factors as temperature, running frequency and the design [17]. The dynamic power dissipation is caused by signal transitions in the circuit. The first factor affecting the dynamic power dissipation is the effective capacitance, namely, the sum of the parasitic effects due to interconnections and transistors, and the emulated capacitance due to short-circuit. The second factor is the switching activity, which is represented by the average toggle rate at each clock.

The power consumptions of the designs are evaluated using the tool Virtex-II power estimator, provided by Xilinx. This estimator sets the core supply voltage to be 1.5 V, the I/O supply

voltage to be 3.3 V, and the supply voltage for the auxiliary circuits to be 3.3 V, conforming to the requirements of Virtex-II XC2V1000-4FG256 FPGA logic. Comparison between the two systolic array-based decoders in terms of power consumption is done by focusing on the dynamic power dissipation in the CLB logic used by the two decoders, since the dynamic power consumption is dominant in the design and is affected by the average toggle rates of the components. In the power estimation, the amount of hardware resource used and the frequency of the system for both the nonadaptive and the adaptive Viterbi decoders are based on the results from Xilinx Design Manager after post synthesis. The frequency of the system is confirmed after prototyping the design to the FPGA chip. The components for each systolic array are activated at every clock cycle and their outputs assumed to appear in random pattern, since the assumption would lead to the worst case estimate of the toggle rate at the output. The switching activities of the n -bit random outputs gen-

TABLE III
BLOCK SELECTRAM POWER ESTIMATOR RESULTS

Module	Block Select-RAM	Port A				Port B				V_{CCint} Subtotal (mW)
		Freq. (MHz)	Width	Read Rate %	Write Rate %	Freq. (MHz)	Width	Read Rate %	Write Rate %	
ROM	1	40	18	100%	0%	0	0	0%	0%	2
ROM0	1	40	18	2%	0%	0	0	0%	0%	0
ROM1	1	40	18	2%	0%	0	0	0%	0%	0
ROM2	1	40	18	2%	0%	0	0	0%	0%	0
ROM3	1	40	18	2%	0%	0	0	0%	0%	0
RAM0	1	40	18	0%	2%	40	18	100%	0%	2
RAM1	1	40	18	0%	2%	40	18	100%	0%	2
RAM2	1	40	18	0%	2%	40	18	100%	0%	2
RAM3	1	40	18	0%	2%	40	18	100%	0%	2
RAM1024_0	1	40	9	0%	2%	40	9	6%	0%	0
RAM1024_1	1	40	9	0%	2%	40	9	6%	0%	0
RAM1024_2	1	40	9	0%	2%	40	9	6%	0%	0
RAM1024_3	1	40	9	0%	2%	40	9	6%	0%	0
Total									15	

TABLE IV
PERFORMANCE OF NONADAPTIVE AND ADAPTIVE VITERBI DECODERS

Decoder	Clock Frequency (MHz)	Speed (Kbps)	Slice	Static Power (mW)	Dynamic Power (mW)	Total Power (mW)	Coding Gain in dB at a Bit-Error-Rate of 10^{-5}
Non-Adaptive Viterbi Decoder	40	78	2793	150	677 *	827	5
Adaptive Viterbi Decoder	40	78	2934	150	323 +	473	5

* Includes a power consumption of 33 mW from auxiliary circuits, 15 mW from Block SelectRAM (Table III), 3 mW from I/O cells and DCM, and 626 mW from CLB logic (Table II).

+ Includes a power consumption of 33 mW from auxiliary circuits, 15 mW from Block SelectRAM (Table III), 3 mW from I/O cells and DCM, and 272 mW from CLB logic (Table II).

erated from a component can be reflected by the outputs of an n -bit LFSR, and it is feasible to count the toggles of the outputs for an n -bit LFSR based on VHDL simulation. Thus, the average toggle rates for the components of each systolic array are obtained by computing the average toggle rates of LFSRs with the same data lengths as the corresponding components based on simulation results. In addition, the average toggle rates for general designs provided by the estimator are also used in this estimation. In the adaptive Viterbi decoder, the threshold value $T = -7$ is used (see Section III for the reason for the choice of $T = -7$).

Table II shows the average toggle rate and P_{INT} in different modules of the CLB logic utilized by the two decoders. It is seen from this table that the average toggle rates in the components of array processors, acsPEX16, brcPEX16, and brcREGx16, for

the proposed decoder are only about 30% of the corresponding rate in the nonadaptive decoder. This is due to the fact that the spurious toggles have been eliminated in the proposed decoder, as explained in Section IV-A-2. This results in a reduction in the dynamic power consumed in these three units in the proposed decoder as compared to the nonadaptive one. As can be seen from Table II, the dynamic power consumed by these 3 units in the proposed decoder is only about 35% of that in the nonadaptive decoder. As a consequence, the total internal power P_{INT} in the CLB logic used in the proposed decoder is about 43.7% of that in the nonadaptive decoder.

Table III shows the power estimator results in block select RAMs used in the two decoders. The RAMs for storing the updated path metrics and the states of the survived paths at each stage are accessed by write operations at every 256 clock cycles

instead of at each clock cycle. This is due to the fact that for $K = 9$, there are 256 paths entering a state at a given stage that have to be updated and compared. Therefore, the rate of write operations of the RAMs is very low. Similarly, the ROMs for storing the codewords have a very low rate for the read operations, since these operations are performed at every 256 clock cycles. Consequently, there is very little dynamic power dissipated in these memory units. In addition, the amount of power dissipation in the input–output (I/O) cells and the DCM are only about 2 mW and 1 mW, respectively.

The performance of the two decoders is summarized in Table IV. Clearly, the adaptive Viterbi decoder can operate at the same decoding speed and provide the same coding gain as the Viterbi decoder. However, the total power consumption in the adaptive Viterbi decoder is only 57% of that in the nonadaptive decoder; at the same time, the increase in the hardware is only 5%.

Even though the results concerning the power consumption are theoretical estimates and hence may not be as precise as the actual test results conducted on the FPGA chips implementing the designs, these results are good estimates of the savings in the power consumption achieved from the proposed design, since the theoretical estimates have been obtained based on the FPGA implementations of the two decoders.

VI. CONCLUSION

In this paper, an adaptive Viterbi algorithm based on the strongly connected trellis decoding of binary convolutional codes has been presented. The proposed algorithm is very suitable for a systolic array architecture-based hardware implementation. An extensive simulation study has been carried out in order to show the performance and the efficiency of the proposed algorithm. It has been shown that with a threshold value of $T = -5$, (T being the parameter used in the selection of the survivor paths at a given trellis stage), the proposed algorithm can yield the same error performance as that of the nonadaptive Viterbi algorithm based on a strongly connected trellis decoding, but the amount of the ACS computations is reduced to 42.8%. It has also been shown that this reduction in the ACS computations can be lowered to as low as 30.1% for a threshold value of -7 , with a negligible degradation in the error performance. In the proposed systolic array-based adaptive Viterbi architecture, time multiplexing has been employed to implement a large-constraint-length Viterbi decoder with four pairs of arrays of fixed size. Arithmetic pipelining has been exploited to ensure that the array processors with different processing times can be synchronized without sacrificing the speed of the decoder. A scheme for avoiding large clock-to-data skews and for providing a tolerance to low clock-to-data skews have been investigated to ensure that the timings in the design are not violated. Moreover, a process of spurious toggle reduction has been introduced to eliminate the unnecessary switching activities in the array processors caused by the path metric updates corresponding to the paths that have not survived. The power estimations have been done for both a nonadaptive and

the proposed adaptive Viterbi decoders. The power estimator results have shown that the total power consumption in the adaptive Viterbi decoder is only 57% of that in the nonadaptive one at the expense of a 5% overhead in the hardware.

REFERENCES

- [1] G. D. Forney Jr., "The Viterbi algorithm," *Proc. IEEE*, vol. 61, no. 3, pp. 268–278, Mar. 1973.
- [2] V. K. Gary, *Wireless Network Evolution 2G to 3G*. New York: Prentice-Hall, 2002.
- [3] B. Pandita and S. K. Roy, "Design and implementation of a Viterbi decoder using FPGAs," in *Proc. 12th Int. Conf. VLSI Design*, Jan. 1999, pp. 611–614.
- [4] J.-H. Park and Y.-C. Rho, "Performance test of Viterbi decoder for wide-band CDMA system," in *Proc. IEEE ASP DAC'97*, 1997, pp. 19–23.
- [5] I. Kang and A. N. Willson Jr., "Low-power Viterbi decoder for CDMA mobile terminals," *IEEE J. Solid-State Circuits*, vol. 33, no. 3, pp. 473–482, Mar. 1998.
- [6] Y.-N. Chang, H. Suzuki, and K. K. Parhi, "A 2-Mb/s 256-state 10-mW rate-1/3 Viterbi decoder," *IEEE J. Solid-State Circuits*, vol. 35, no. 6, pp. 826–834, Jun. 2000.
- [7] C.-Y. Chang and K. Yao, "Systolic array processing of the Viterbi algorithm," *IEEE Trans. Inf. Theory*, vol. 35, no. 1, pp. 76–86, Jan. 1989.
- [8] C. G. Caraiscos and K. Z. Pekmestzi, "Low-latency bit-parallel systolic VLSI implementation of FIR digital filters," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 43, no. 7, pp. 529–537, Jul. 1996.
- [9] V. Boriakoff, "FFT computation with systolic arrays, a new architecture," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 44, no. 4, pp. 278–284, Apr. 1994.
- [10] I. K. Proudler, J. G. McWhirter, M. Mooner, and G. Hekstra, "Formal derivation of a systolic array for recursive least squares estimation," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 43, no. 3, pp. 247–254, Mar. 1996.
- [11] M. Guo, M. O. Ahmad, M. N. S. Swamy, and C. Wang, "A low-power systolic array-based adaptive Viterbi decoder and its FPGA implementation," in *Proc. IEEE Int. Symp. Circuits Syst.*, vol. 2, Bangkok, Thailand, May 2003, pp. 256–259.
- [12] F. Chan and D. Haccoun, "Adaptive Viterbi decoding of convolutional codes over memoryless channels," *IEEE Trans. Commun.*, vol. 45, no. 11, pp. 1389–1400, Nov. 1997.
- [13] S. Swaminathan, R. Tessier, D. Goeckel, and W. Burleson, "A dynamically reconfigurable adaptive Viterbi decoder," in *Proc. FPGA'02*, Monterey, CA, Feb. 24–26, 2002.
- [14] R. Henning and C. Chakrabarti, "Low-power approach for decoding convolutional codes with adaptive Viterbi algorithm approximations," in *Proc. ISLPED'02*, Monterey, CA, Aug. 12–14, 2002.
- [15] C. B. Shung, P. H. Siegel, G. Ungerboeck, and H. K. Thaper, "VLSI architecture for metric normalization in the Viterbi algorithm," in *Proc. Int. Conf. Commun.*, vol. 4, Atlanta, GA, Apr. 1990, pp. 1723–1728.
- [16] P. J. Black and T. H. Meng, "A 140-Mb/s, 32-state, radix-4 Viterbi decoder," *IEEE J. Solid-State Circuits*, vol. 27, no. 12, pp. 1877–1885, Dec. 1992.
- [17] L. Shang, A. S. Kaviani, and K. Bathala, "Dynamic power consumption in Virtex-II FPGA family," in *Proc. FPGA'02*, Monterey, CA, Feb. 24–26, 2002.



Man Guo received the B.Eng. degree in industrial processing control from Huazhong University of Science and Technology, Wuhan, China, in 1988 and the Master's degree in electrical and computer engineering from Concordia University, Montreal, QC, Canada, in 2002.

From 1988 to 1993, she worked as an Engineer designing hardware and software for embedded systems at Kaida Computer Corp., Ltd., Wuhan, China, and from 1993 to 1999 at Turing Digital Technology Ltd., Wuhan, China. She is now an ASIC Designer

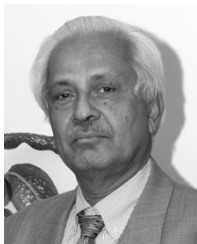
with 1021 Technologies, Kanata, ON, Canada.



M. Omaid Ahmad (S'59–M'78–SM'83–F'01) received the B. Eng. degree from Sir George Williams University, Montreal, QC, Canada, and the Ph. D. degree from Concordia University, Montreal, QC, Canada, both in electrical engineering.

From 1978 to 1979, he was a member of the Faculty of the New York University College, Buffalo. In September 1979, he joined the Faculty of Concordia University, where he was an Assistant Professor of Computer Science. Subsequently, he joined the Department of Electrical and Computer Engineering of the same university, where presently he is Professor and Chair of the Department. He is a Researcher in the Micronet National network of Centers of Excellence and was previously an Examiner of the Order of Engineers of Quebec. He has published extensively in the area of signal processing and holds three patents. His current research interest includes the area of multidimensional filter design, image and video signal processing, nonlinear signal processing, communication digital signal processing, artificial neural networks, and very large-scale integration circuits for signal processing.

Dr. Ahmad was an Associate Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: FUNDAMENTAL THEORY AND SYSTEMS from June 1999 to December 2001. He was the Local Arrangements Chairman of the 1984 IEEE International Symposium on Circuits and Systems (CAS). During 1988, he was a member of the Admission and Advancement Committee of the IEEE. Presently, he is the Chairman of the IEEE CAS Chapter (Montreal Section). He is a recipient of the Wighton Fellowship from the Sanford Fleming Foundation.



M. N. S. Swamy (S'59–M'62–SM'74–F'80–LF'01) received the B.Sc. (Hons.) degree in mathematics from Mysore University, Mysore, India, in 1954, the Diploma in electrical communication engineering from the Indian Institute of Science, Bangalore, India, in 1957, and the M.Sc. and Ph. D. degrees in electrical engineering from the University of Saskatchewan, Saskatoon, SK, Canada, in 1960 and 1963, respectively. In August 2001 he was awarded a Doctor of Science in Engineering (Honoris Causa) by Ansted University "In recognition of his exemplary contributions to the research in Electrical and Computer Engineering and to Engineering Education, as well as his dedication to the promotion of Signal Processing and Communications Applications."

He is presently a Research Professor and the Director of the Center for Signal Processing and Communications in the Department of Electrical and Computer Engineering at Concordia University, Montreal, QC, Canada, where he served as the Chair of the Department of Electrical Engineering from 1970 to 1977, and Dean of Engineering and Computer Science from 1977 to 1993. Since July 2001, he holds the Concordia Chair (Tier I) in Signal Processing. He has also taught in the Electrical Engineering Department of the Technical University of Nova Scotia, Halifax, NS, Canada and the University of Calgary, Calgary, AB, Canada, as well as in the Department of Mathematics at the University of Saskatchewan. He has published extensively in the areas of number theory, circuits, systems and signal processing, and holds four patents. He is the co-author of two book chapters and three books: *Graphs, Networks and Algorithms* (New York: Wiley, 1981), *Graphs: Theory and Algorithms* (New York: Wiley, 1992), and *Switched Capacitor Filters: Theory, Analysis and Design* (London, U.K.: Prentice-Hall International UK Ltd., 1995). A Russian translation of the first book was published by Mir Publishers, Moscow, U.S.S.R., in 1984, while a Chinese version was published by the Education Press, Beijing, in 1987. He is a member of Micronet, a National Network of Centers of Excellence in Canada, and also its coordinator for Concordia University.

Dr. Swamy is the Past President of the IEEE Circuits and Systems (CAS) Society. He has served the IEEE CAS Society in various capacities such as Vice President (Publications) during 2001–2002 and as Vice-President in 1976, Editor-in-Chief of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: FUNDAMENTAL THEORY AND SYSTEMS during 1999–2001, Associate Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS during 1985–1987, Program Chair for the 1973 IEEE CAS Symposium, General Chair for the 1984 IEEE CAS Symposium, Vice-Chair for the 1999 IEEE CAS Symposium and a member of the Board of Governors of the CAS Society. He is the recipient of many IEEE-CAS Society awards including the Education Award in 2000, Golden Jubilee Medal in 2000, and the 1986 Guillemin–Cauer Best Paper Award. He is a Fellow of the Institute of Electrical Engineers (U.K.), the Engineering Institute of Canada, the Institution of Engineers (India), and the Institution of Electronic and Telecommunication Engineers (India).



Chunyan Wang (SM'04) received the B.Eng. degree in electronics from Jiao Tung University, Shanghai, China, in 1982, and the M.Eng. and Ph.D. degrees from Université Paris Sud, Paris, France, in 1992 and 1996, respectively.

She joined Concordia University, Montreal, QC, Canada, in 1997 as an Assistant Professor, where she is presently an Associate Professor of Electrical and Computer Engineering and Concordia Research Chair in low-power very large-scale integration (VLSI) circuits. Her current research areas are low-power analog-mixed VLSI design, CMOS sensor integration, and VLSI implementation of digital signal processing systems. She holds a number of patents in low-power VLSI designs.